

# Phỏng vấn CDC Global

## 12 câu & đáp án

Nguyen Ngoc Minh Thy · 25/08/2026  
Vị trí: Senior AI / LLM Engineer

| Nhóm câu | Nội dung                                                                                  |
|----------|-------------------------------------------------------------------------------------------|
| 1–3      | Chẩn đoán RAG: recall@k cao mà sai · abstain khi context rỗng · invalidate khi schema đổi |
| 4, 12    | Guardrail cho agent: quyền gọi API · chọn sai tool trong 10 tool                          |
| 5–6      | LLM-as-judge: gap 95% judge vs 60% user · cơ chế xử lý bất đồng                           |
| 7–10     | Dự án Fabrion: model theo vai trò · LangChain vs tự viết · 0.14 → 0.93 · 7 scorer         |
| 11       | Case study: isolation cho 100 khách hàng                                                  |

## Phần 7 — Phỏng vấn CDC Global (25/08/2026): 12 câu và đáp án

---

Đây là 12 câu interviewer CDC Global đã hỏi. Mỗi câu gồm: **interviewer đang test gì** → **đáp án 60–90 giây** → **follow-up sẽ bị hỏi tiếp**.

Trục xuyên suốt vẫn là định vị của bạn: *"tôi làm ở phần quyết định xem một agent có đáng tin không, trước khi có người dựa vào nó."* Sáu câu đầu là câu hỏi hệ thống — kéo về **đo lường**. Sáu câu sau là câu hỏi về dự án — kéo về **bất biến hai tầng**.

---

**Bảng tra nhanh — câu chốt của từng câu**

| #  | Câu hỏi                       | Câu chốt phải nói được                                                                             |
|----|-------------------------------|----------------------------------------------------------------------------------------------------|
| 1  | recall@k cao mà answer sai    | "recall@k đo ở mức chunk id, generator cần mức answer span — hai cái đó lệch nhau"                 |
| 2  | Context rỗng/không liên quan  | "Abstain là một outcome được thiết kế và được đo, không phải fallback xấu hổ"                      |
| 3  | Schema đổi, field biến mất    | "Re-index là tối ưu hoá; <b>runtime validation mới là lớp bắt buộc</b> "                           |
| 4  | Agent gọi API nguy hiểm       | "Autonomy nằm ở <i>chọn làm gì</i> , không nằm ở <i>được phép làm gì</i> "                         |
| 5  | Judge 95% vs user 60%         | "95% per-turn qua 8 turn = 66% per-session. Đơn vị đo lệch trước khi rubric lệch"                  |
| 6  | 2 judge bất đồng              | "Bất đồng là tín hiệu rubric hỏng, không phải bài toán bỏ phiếu"                                   |
| 7  | Model nào cho phần nào        | "Tôi chọn model theo <b>vai trò trong harness</b> ; judge không bao giờ cùng family với generator" |
| 8  | LangChain hay tự viết         | "Framework ở chỗ tiết kiệm thời gian; tự viết ở chỗ cần kiểm soát tuyệt đối"                       |
| 9  | Giới hạn LangChain multi-step | "Recovery controller của tôi vốn đã là state machine viết tay — và tôi cần nó deterministic"       |
| 10 | 0.14 → 0.93, 7 scorer         | "Phần lớn mức tăng đến từ <b>ground vào semantic layer</b> , không phải từ đổi model"              |
| 11 | 100 tenant isolation          | "Isolation là bất biến phải đúng ở 5 chỗ; chỗ yếu nhất quyết định"                                 |
| 12 | 10 tool, chọn sai tool        | "Tool description chính là prompt. Đây là bài toán đo lường, không phải prompt engineering"        |

# Bối cảnh — 12 bộ chuẩn dữ liệu cốt lõi của CES Global

Đây là cách team CES Global làm việc: **xây 12 bộ chuẩn về dữ liệu TRƯỚC khi xây nền tảng IT + AI cho doanh nghiệp**. Đọc phần này trước 12 câu hỏi — nó giải thích *vì sao* họ hỏi đúng những câu đó, và cho bạn từ vựng chung để nói chuyện với họ.

## 12 bộ chuẩn

1. **Master Data** — Chuẩn hóa các danh mục dùng chung: sản phẩm, khách hàng, nhà cung cấp, kho, nhân sự, đơn vị tính, địa điểm, kênh bán...
2. **Business Glossary & Data Dictionary** — Thống nhất thuật ngữ nghiệp vụ và định nghĩa rõ từng trường dữ liệu: tên, ý nghĩa, kiểu dữ liệu, giá trị hợp lệ, nguồn chuẩn.
3. **Template Standard** — Chuẩn hóa biểu mẫu, file nhập liệu, cấu trúc cột, validation và quản lý phiên bản.
4. **Business Rule Register** — Ghi lại điều kiện, công thức tính, ngoại lệ, thẩm quyền phê duyệt và ngày hiệu lực.
5. **Mapping Register** — Quản lý việc ánh xạ mã cũ, mã đối tác, mã từ các hệ thống khác về một mã chuẩn.
6. **Data Ownership & Stewardship** — Mỗi dữ liệu cần xác định rõ: ai sở hữu, ai chịu trách nhiệm cập nhật, ai phê duyệt.
7. **Data Quality Standard** — Đặt tiêu chuẩn về độ đúng, đủ, duy nhất, nhất quán, hợp lệ và kịp thời của dữ liệu.
8. **Metadata & Data Catalog** — Doanh nghiệp phải biết mình đang có dữ liệu gì, nằm ở đâu, thuộc phòng ban/domain nào và ai quản lý.
9. **Data Lineage & Provenance** — Biết dữ liệu bắt nguồn từ đâu, đã qua những bước biến đổi nào và cuối cùng được sử dụng ở đâu.
10. **Data Classification & Access** — Phân loại dữ liệu và xác định rõ quyền xem, sửa, tải xuống, chia sẻ hoặc cung cấp cho AI.
11. **Data Lifecycle & Retention** — Quy định vòng đời dữ liệu từ lúc tạo ra, sử dụng, lưu trữ, archive cho đến khi xóa bỏ.
12. **Data Contract & Integration** — Chuẩn hóa cách các hệ thống trao đổi dữ liệu với nhau: schema, API, version, SLA và cách xử lý lỗi.

## Điểm quan trọng nhất

**Chuẩn hóa dữ liệu không phải dự án của riêng phòng IT. Đó là bài toán phối hợp giữa nghiệp vụ + dữ liệu + công nghệ + quản trị.**

Mỗi dữ liệu quan trọng cần trả lời được ít nhất sáu câu:

**Đây là dữ liệu gì? → Ai chịu trách nhiệm? → Nguồn nào là chuẩn? → Quy tắc nào áp dụng? → Phiên bản nào đang hiệu lực? → Ai được phép sử dụng?**

Khi nền tảng này hình thành, doanh nghiệp mới tiến được tới bước tiếp theo: **Data → Knowledge → AI Agent**. Khi đó AI Agent không chỉ "đọc được dữ liệu", mà biết được: nguồn nào đáng tin → dữ liệu nào mới nhất → quy định nào phải tuân thủ → mình được phép làm gì → **trường hợp nào phải chuyển cho con người phê duyệt**.

Đây cũng là nền tảng để doanh nghiệp từng bước xây dựng một **Enterprise Knowledge Base / Knowledge Fabric** — nơi dữ liệu và tri thức từ ERP, CRM, tài liệu, quy trình, email và các hệ thống nội bộ được chuẩn hóa để **con người và AI Agent cùng sử dụng**.

Map sang 12 câu phỏng vấn — vì sao họ hỏi như vậy

Đọc bảng này thì thấy rõ: họ không hỏi câu hỏi RAG chung chung, họ hỏi đúng những chỗ mà 12 bộ chuẩn của họ chạm vào AI agent. Trả lời bằng từ vựng của họ sẽ nghe như người trong nhà.

| Bộ chuẩn                                      | Câu hỏi liên quan                      | Điểm nổi                                                                                                 |
|-----------------------------------------------|----------------------------------------|----------------------------------------------------------------------------------------------------------|
| 2 · Business Glossary & Data Dictionary       | Câu 3 (schema đổi), Câu 10 (0.14→0.93) | Chính là <b>Cube semantic layer</b> của bạn — metric định nghĩa một lần, agent không được định nghĩa lại |
| 4 · Business Rule Register (có ngày hiệu lực) | Câu 1 (distractor "gần đúng"), Câu 3   | Công thức đổi theo thời gian → phải có metadata thời gian, nếu không agent trộn quy tắc cũ với mới       |
| 6 · Data Ownership & Stewardship              | Câu 4 (guardrail API)                  | "Ai phê duyệt" chính là <b>human-in-the-loop ở ranh giới không đảo ngược</b>                             |
| 7 · Data Quality Standard                     | Câu 10 (7 scorer), Câu 2 (abstain)     | Đúng/đủ/duy nhất/nhất quán/hợp lệ/kịp thời — đây là <b>deterministic scorer</b> ở tầng dữ liệu           |
| 8 · Metadata & Data Catalog                   | Câu 3 (invalidate)                     | Catalog là <b>source of truth cho fingerprint</b> , không phải vector index                              |
| 9 · Data Lineage & Provenance                 | Câu 3, Câu 2 (grounding)               | Lineage cho biết đổi 1 field thì phải re-index chunk nào; provenance chính là <b>evidence span</b>       |
| 10 · Data Classification & Access             | <b>Câu 11 (100 tenant)</b> , Câu 4     | "Quyền cung cấp cho AI" — chính xác là bài toán isolation + tenant resolve từ auth token                 |
| 12 · Data Contract & Integration              | Câu 3, Câu 4, Câu 12                   | Schema/version/SLA/xử lý lỗi = <b>contract validation trước khi execute</b> , và tool schema strict      |

Ba câu nên nói ra trong vòng sau — chúng nổi thẳng việc bạn đã làm vào ngôn ngữ của họ:

- "Cách tiếp cận 'chuẩn hóa trước, AI sau' đúng với thứ tôi đã trải qua: mức tăng lớn nhất từ 0.14 lên 0.93 không đến từ đổi model, mà đến từ ground agent lên semantic layer với metric đã định nghĩa — tức là bộ chuẩn số 2 của các anh, áp dụng cho agent."
- "Sáu câu hỏi các anh yêu cầu mỗi dữ liệu phải trả lời được — tôi coi đó là contract chạy được, không phải tài liệu. Ở hệ thống của tôi, 'phiên bản nào đang hiệu lực' là `schema_fingerprint`, và 'ai được phép sử dụng' là tenant bind qua auth token chứ không phải argument LLM điển."
- "Về 'trường hợp nào phải chuyển cho con người phê duyệt' là chỗ tôi có kinh nghiệm đo được: tôi giữ autonomy log — mỗi lần người can thiệp thì phân loại one-off hay structural gap. Nhờ đó autonomy mở dần dựa trên bằng chứng, không phải dựa trên niềm tin."

**Rủi ro cần lường trước:** một tổ chức đặt 12 bộ chuẩn lên trước thường **chưa có golden set và regression gate cho agent** — họ có kỷ luật ở tầng dữ liệu, nhưng tầng agent thì mới. Đó chính là khoảng trống bạn lấp được, và là lý do nên hỏi ngược câu 1 và câu 2 ở cuối bài.

# Câu 1 — recall@k cao nhưng answer sai: lỗi ở reranking, context construction hay generation?

**Interviewer đang test:** bạn cô lập biến số hay bạn đoán mò. Đây là câu lọc ứng viên mạnh.

## Đáp án

Trước hết phải nói tại sao đây **không phải nghịch lý**: `recall@k` chỉ trả lời "*chunk vàng có nằm đâu đó trong top-k không*", đo ở mức **chunk id**. Generator cần thứ khác — **answer span có nằm trong prompt cuối cùng không**. Hai cái này lệch nhau rất nhiều. `recall@k` không đo:

| recall@k không đo               | Metric thay thế                         | Triệu chứng                                                      |
|---------------------------------|-----------------------------------------|------------------------------------------------------------------|
| Vị trí chunk vàng               | precision@1-3, nDCG, MRR                | rank 18/20 → lost-in-the-middle                                  |
| Tỉ lệ nhiễu                     | precision@k, số distractor              | hallucinate, nhất là distractor "gần đúng" (sai kỳ, sai version) |
| Answer span có nguyên vẹn không | <b>answer-in-context@k</b> (span-level) | trả lời cụt hoặc null                                            |
| Chunk boundary                  | boundary check                          | đáp án bị cắt đôi                                                |

Sau đó tôi chạy một **ladder chẩn đoán**, không đoán:

```
0. Log ĐÚNG chuỗi prompt cuối cùng prod gửi đi (sau rerank, sau truncate)
   → debug cái model thực sự nhìn thấy, không phải cái mình tưởng nó thấy

1. Answer span có tồn tại nguyên vẹn trong corpus không? (grep / BM25 exact)
   Không → ingest/parsing bug (PDF bẻ nát, OCR sai, chunk cắt đôi). Dừng, sửa ingest.

2. Nó có trong prompt cuối không? (answer-in-context@k, span-level)
   Không, nhưng có trong top-k thô → rerank hoặc truncation đã giết nó
   Không, và cũng không trong top-k → retrieval

3. Oracle test: chỉ chunk vàng + PROMPT PROD THẬT
   Vẫn sai → chunking hoặc prompt (KHÔNG phải rank)
   Đúng → sang bước 4

4. Full k chunks + prompt prod thật
   Sai → nhiễu / lost-in-the-middle → CONTEXT CONSTRUCTION + rerank + giảm k
   Đúng → bug ở tầng ghép prompt hoặc lệch config giữa test và prod

5. Đo lại trên eval set. Không có số thì không gọi là fix.
```

Ba nhánh kết luận map thẳng vào ba khả năng interviewer đưa ra:

- **Bước 2 fail nhưng top-k thô có → reranking.** Dấu hiệu định lượng: `recall@50` cao, `precision@5` thấp.
- **Bước 3 pass, bước 4 fail → context construction.** Chunk vàng có đó nhưng bị nhiễu dìm.
- **Bước 3 fail → chunking hoặc generation.** Phân biệt: chunk vàng đọc bằng mắt có đủ thông tin trả lời không? Đủ mà model vẫn sai → generation/grounding. Không đủ → chunking.

## Follow-up chắc chắn bị hỏi

"Dùng MMR để lọc và sắp xếp lại thì sao?"

MMR là thuật toán **chọn tập con để giảm trùng lặp**, không phải thuật toán ước lượng độ liên quan:

```
MMR = argmax [  $\lambda \cdot \text{sim}(q, d) - (1-\lambda) \cdot \max \text{sim}(d, d_{\text{selected}})$  ]
```

Nó chỉ tái sắp xếp *trong* danh sách bi-encoder đã chấm. **Embedding chấm sai thứ tự thì MMR không kéo tài liệu đúng lên được** — nó chỉ chọn một tập con đa dạng của một danh sách sai.

- **Có ích khi:** top-k bị chi phối bởi chunk gần trùng (boilerplate lặp, doc duplicate, overlap chồng lấn) đẩy chunk vàng ra khỏi budget.
- **Gây hại khi:** đáp án cần nhiều chunk liên kề và *giống nhau* (quy trình dài, bảng bị chunk thành nhiều mảnh) — MMR chủ động loại chúng vì cho là dư thừa. Với câu tra cứu 1 sự kiện, diversity = bươm nhiều.

Đặt đúng chỗ: hybrid retrieve (BM25+dense, RRF) → cross-encoder rerank → MMR/dedup trên top~20,  $\lambda \approx 0.7-0.8 \rightarrow 5-8$  chunk. MMR chạy **trên điểm reranker**, không phải cosine thô. Nếu chỉ được chọn một thứ: **cross-encoder reranker**, mạnh hơn MMR nhiều cho bài toán thứ tự sai.

**"Fix theo thứ tự nào?"** → (1) đo answer-in-context@k, (2) cross-encoder rerank + giảm k (20→6), (3) hybrid BM25+dense, (4) contextual retrieval (prepend 1-2 câu ngữ cảnh doc-level vào mỗi chunk trước khi embed), (5) parent-document retrieval, (6) metadata filter theo thời gian/version.

---

## Câu 2 — Retrieval trả về context không liên quan hoặc rỗng: NL-to-SQL agent và Document Extract Agent xử lý thế nào? LLM nói "không biết" hay vẫn cố generate?

**Interviewer đang test:** bạn có coi abstain là một thiết kế hay chỉ là fallback. Và bạn có hiểu **asymmetric cost** trong analytics không.

### Đáp án

Nguyên tắc: **abstain là một outcome hợp lệ, được thiết kế và được đo.** Trong analytics và document extraction, một con số sai tệ hơn nhiều một câu "tôi không biết" — vì số sai đi vào báo cáo và không ai kiểm lại. Nên hệ thống của tôi **fail-closed**.

Quan trọng hơn: tôi **không để LLM tự quyết định có nên abstain hay không**. Đó là quyết định của code, ở tầng trước generation.

Ba mức, quyết bằng code:

| Mức  | Điều kiện                              | Hành vi                                                                           |
|------|----------------------------------------|-----------------------------------------------------------------------------------|
| Rỗng | không có candidate nào qua ngưỡng      | <b>Không gọi LLM.</b> Trả clarifying question hoặc "không có dữ liệu cho câu này" |
| Thấp | top score / reranker score dưới ngưỡng | Gọi LLM nhưng <b>bắt buộc kèm citation</b> , gắn cờ low-confidence lên UI         |
| Đủ   | qua ngưỡng                             | Generate bình thường, vẫn kèm citation                                            |

Với **NL-to-SQL agent** — và đây là chỗ kiến trúc của tôi khác đa số:

Agent của tôi **không tự viết SQL tự do; nó ground lên Cube semantic layer với metric đã được định nghĩa.** Hệ quả trực tiếp: "retrieval không liên quan" biến thành một lỗi **resolve được bằng code, không cần LLM tự nhận**. Luồng là: resolve entity/metric từ câu hỏi → map vào Cube metadata → nếu không map được field/measure nào thì **dừng ngay, không sinh query**, trả về câu hỏi làm rõ kèm danh sách metric gần nhất. Không bao giờ có chuyện agent bịa ra tên cột.

Đây chính là cái loại bỏ nguyên một lớp câu trả lời sai: join sai, grain sai, định nghĩa metric sai. Vector store không làm được điều đó.

Với **Document Extract Agent** — abstain ở **mức field**, không phải mức document:

Mỗi field trả về {value, evidence\_span, confidence}. Không có evidence span → value = null kèm reason code (NOT\_FOUND / AMBIGUOUS / UNPARSEABLE), không đoán. Sau đó một **groundedness check deterministic**: giá trị trích ra phải xuất hiện thật trong nguồn (khớp literal hoặc khớp sau khi chuẩn hoá số/ngày). Không khớp → loại, dù LLM tự tin đến đâu.

Đây đúng là việc tôi đã làm ở nhánh Document AI: 4 PDF reader × 3 phương pháp, reference set tự kiểm tay, và tiêu chí chính là groundedness — *giá trị này có thật trong nguồn không* — chứ không phải "câu trả lời nghe có hợp lý không".

Và phần quan trọng nhất — đo nó:

Tôi có **unanswerable set**: những câu mà dữ liệu không trả lời được. Metric không phải pass rate mà là **false-answer rate** (tỉ lệ agent trả lời tự tin cho câu đáng lẽ phải abstain) và **over-abstain rate** (tỉ lệ câu trả lời được). Cộng thêm **null control** và **cheater control** — cả hai phải 0 điểm. Nếu agent không bao giờ nói "không biết" trên bộ unanswerable thì grader của bạn không có răng, và pass rate của bạn vô nghĩa.

## Follow-up

"**Ngưỡng confidence đặt bao nhiêu?**" → Không có con số phổ quát. Tôi calibrate trên eval set: vẽ đường false-answer rate và over-abstain rate theo ngưỡng, chọn điểm theo **cost bất đối xứng của domain đó**. Với báo cáo tài chính tôi chấp nhận over-abstain cao. Và tôi đo lại ngưỡng mỗi khi đổi embedding model — score không so sánh được giữa các model.

---

## Câu 3 — Semantic layer đổi schema, retrieval point vào field không tồn tại. Có invalidate tự động không hay chạy lại thủ công?

---

**Interviewer đang test:** bạn có hiểu rằng đây là bài toán **data contract**, không phải bài toán vector index. Câu này rất sát với nền tảng dữ liệu bạn đã tự xây.

### Đáp án

Câu trả lời ngắn: **có cơ chế tự động, nhưng tôi không bao giờ tin vào nó một mình**. Vì luôn tồn tại cửa sổ thời gian giữa lúc deploy schema và lúc index bắt kịp. Nên tôi làm hai lớp, và lớp bắt buộc là lớp thứ hai.

#### Lớp 1 — Invalidation theo fingerprint (tự động, event-driven)

Source of truth là `dbt manifest.json` + Cube meta API, **không phải vector index**. Mỗi lần `dbt run` / cube deploy, tôi sinh một `schema_fingerprint` = hash của (tên model, tên field, kiểu, định nghĩa measure). Index lưu kèm fingerprint lúc embed. Lỗi fingerprint → index bị đánh dấu **stale**.

Trigger là event, không phải cron: Dagster post-hook sau khi materialize → emit event → re-embed **chỉ những model đã đổi** (incremental, không full reindex). OpenMetadata column-level lineage cho biết đổi một field ở mart thì ảnh hưởng cube nào, chunk nào — nên phạm vi re-index nhỏ và xác định được.

#### Lớp 2 — Runtime validation trước khi execute (bắt buộc)

Đây mới là lớp không thể bỏ. Query do agent sinh ra được **validate với Cube meta trước khi chạy**:

`cubejs_dry_run` / `clickhouse_dry_run`. Field không tồn tại → bắt được **trước khi chạm dữ liệu**, và lỗi đó không đi ra người dùng mà đi vào recovery loop dưới dạng lỗi có cấu trúc: `UNKNOWN_FIELD: revenue_net`, gợi ý: `revenue_net_amount`.

Lý do lớp này bắt buộc: nó **fail-closed và không phụ thuộc index có tươi hay không**. Kể cả khi re-index chưa chạy, agent vẫn không thể trả lời sai — nó chỉ có thể trả lời đúng hoặc báo lỗi. Re-index là tối ưu hoá **trải nghiệm** (agent tìm đúng field nhanh hơn); validation là đảm bảo **tính đúng**.

#### Lớp 3 — Non-regression gate trong CI

Khi schema đổi, chạy lại golden set. Pass rate tụt so với baseline → **chặn deploy**. Đây là thứ biến "schema đổi" từ một sự cố production thành một lần CI đỏ. Với nền tảng của tôi (65 dbt models → 20+ Cube models) thì đây là cách duy nhất giữ được tốc độ thay đổi schema mà không làm hỏng agent.

### Follow-up

**"Nếu không có ai xây mấy cái đó thì sao?"** → Thì mặc định là thủ công, và nó **luôn drift** — không phải có thể, mà chắc chắn, vì không ai nhớ re-index sau mỗi lần đổi cột. Đó chính là lý do tôi đặt validation ở runtime: nó biến một lỗi im lặng (trả lời sai) thành một lỗi ồn ào (query không chạy). Lỗi ồn ào thì sửa được.

---

## Câu 4 — Agent có quyền gọi API. Làm sao ngăn API nguy hiểm/sai mà vẫn cho phép agent tự chủ?

**Interviewer đang test:** bạn có phân biệt được guardrail bằng **code** và guardrail bằng **prompt** không. Đây là câu rất nhiều ứng viên trả lời sai.

### Đáp án

Nguyên tắc gốc: **autonomy nằm ở "chọn làm gì", không nằm ở "được phép làm gì"**. Tôi tách *capability* khỏi *decision*. Agent được tự do quyết định chuỗi hành động; nhưng tập hành động khả thi được định nghĩa bằng code ở tầng harness, và **agent không có cách nào mở rộng nó**.

Câu tôi muốn interviewer nhớ: *"Đừng gọi API xóa" viết trong system prompt không phải là một guardrail — nó là một lời đề nghị*. Guardrail phải nằm ngoài tầm với của model.

### Các lớp, từ ngoài vào trong:

| Lớp                | Cơ chế                                                                                                                                            | Chặn được gì                                                               |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Credential         | Agent <b>không giữ secret</b> . Credential ở vault/gateway, substitute lúc egress. Read role và write role tách biệt                              | Agent không thể gọi thứ nó không được cấp, kể cả khi bị prompt injection   |
| Tool surface       | Allowlist theo phase — 10 tool không có nghĩa là lộ cả 10 mọi lúc                                                                                 | Giảm không gian chọn sai                                                   |
| Schema             | Strict JSON schema, <code>enum</code> thay vì free string, validate <b>trước</b> khi thực thi                                                     | Tham số sai, injection qua argument                                        |
| Phân loại tác động | read → auto · write reversible → auto + log + rollback được · <b>write không đảo ngược hoặc vượt ngưỡng tiền → dry-run hoặc human-in-the-loop</b> | Thiệt hại không cứu được                                                   |
| Dry-run            | <code>cubejs_dry_run</code> , SQL <code>EXPLAIN</code> , sandbox endpoint của API                                                                 | Query/call sai cú pháp hoặc sai phạm vi, trước khi chạm production         |
| Budget             | Trần số call, trần tiền, trần wall-time, stop condition rõ                                                                                        | <b>Loop vô hạn — failure mode phổ biến nhất của agent trong production</b> |
| Idempotency        | Idempotency key cho mọi write                                                                                                                     | Retry gây double-charge / ghi trùng                                        |
| Audit              | Mọi call vào trace: tool, args, token, cost, kết quả — replay được                                                                                | Không debug được nếu thiếu                                                 |

**Và phần ít người nói:** tôi đo chính cái guardrail đó. Ở Fabrion tôi giữ một **autonomy log** — mỗi lần con người phải can thiệp, tôi phân loại *one-off* hay *structural gap*, và mỗi structural gap trở thành một ticket. Guardrail chặn quá nhiều không phải là chuyện đương nhiên phải chịu; đó là tín hiệu thiết kế sai cần sửa. Đây là cách tôi giữ autonomy thật sự tăng theo thời gian thay vì đóng băng ở mức "an toàn nhưng vô dụng".

### Follow-up

**"Human-in-the-loop thì còn gì là tự chủ?"** → HITL đặt ở **ranh giới không đảo ngược**, không đặt ở mọi bước. Agent chạy 20 bước tự do rồi dừng ở 1 bước cần duyệt thì vẫn là tự chủ. Và theo thời gian, autonomy log cho tôi biết bước nào duyệt 50 lần đều OK — bước đó chuyển sang auto kèm rollback. Autonomy được **mở dần dựa trên bằng chứng**, không mở dựa trên niềm tin.

## Câu 5 — LLM-as-judge cho pass 95%, user thật đánh giá ~60%. Chuyện gì đã xảy ra trong evaluation framework?

**Interviewer đang test:** đây là câu hay nhất trong bộ. Nó test xem bạn có hiểu **construct validity** không — tức là "cái tôi đo có phải cái tôi quan tâm không".

### Đáp án

Tôi bắt đầu bằng thứ ít ai kiểm tra trước: **đơn vị đo có khớp không**.

Nếu judge chấm **từng turn** còn user đánh giá **cả session/task**: 0.95 mỗi turn qua 8 turn là  $0.95^8 \approx 0.66$ . Con số 60% rơi gần như chính xác vào đó. **Kiểm tra phép nhân này trước khi động vào rubric** — nhiều lần đây là toàn bộ câu chuyện, và không có gì "hổng" cả ngoài việc hai bên đang báo cáo hai đại lượng khác nhau.

Nếu không phải, tôi soi theo bốn nhóm nguyên nhân:

#### 1. Construct validity — judge chấm sai thứ

Judge chấm "câu trả lời mạch lạc, có trích dẫn, đúng format". User quan tâm "con số này có đúng không, tôi có dùng được không". Judge **không có gold answer** thì nó chỉ chấm được *plausibility*, không chấm được *correctness* — và một câu trả lời sai được viết tự tin sẽ luôn qua.

#### 2. Distribution shift — bộ đề không đại diện traffic

Golden set sạch, đủ ngữ cảnh, một lượt. Câu user hỏi thật thì mơ hồ, thiếu ngữ cảnh, multi-turn, sai chính tả, lẫn tiếng Việt/tiếng Anh, hỏi về data vừa mới đổi. Bộ đề tự soạn hầu như luôn dễ hơn thực tế vì người soạn biết hệ thống làm được gì.

#### 3. Judge bias — lỗi có hệ thống, không phải nhiễu

- **Self-preference:** judge cùng family với generator → thiên vị chính nó
- **Verbosity bias:** câu dài, có bullet, nghe chắc chắn → điểm cao
- **Position bias, sycophancy**
- **Leakage:** gold answer hoặc bộ đề lọt vào context

#### 4. Rubric và control yếu

Rubric pass/fail lỏng, anchor examples toàn ca dễ, thang quá thô. Và nếu chưa có **null control / cheater control** (agent trả lời rỗng và agent gian lận phải bị 0 điểm) thì con số 95% chưa bao giờ được kiểm chứng.

#### Cách tôi xử lý — 5 bước:

1. Lấy **100 ca user đánh fail**, người gán nhãn thủ công, so với judge. Tính **agreement (Cohen's kappa)**, không nhìn pass rate. Judge tốt là judge *đồng ý với người*, không phải judge cho điểm cao.
2. Calibrate rubric trên chính các ca bất đồng, thêm chúng làm anchor example.
3. **Chuyển mọi thứ đo được sang deterministic**. Con số đúng/sai, cột đúng/sai, query chạy được/không — những cái này không cần judge. Judge chỉ nên chấm phần thật sự chủ quan.
4. Đổi golden set sang **sampling từ traffic thật** có stratify, không tự soạn 100%.
5. Đo **per-session**, đo thêm cost/latency, vì user tính cả những thứ đó vào "pass".

#### Câu chốt — kéo về bất biến của tôi:

"Ở LAAF tôi khoá một bất biến: **Tier 1 deterministic quyết PASS/FAIL, LLM judge chỉ advisory, và judge không bao giờ được lật kết quả deterministic**. Chính vì vậy tình huống này không thể xảy ra theo cách

nguy hiểm nhất ở hệ thống của tôi — nếu pass rate 95% của bạn đến từ judge, thì nó chưa bao giờ là ship gate. Grader là source of truth."

---

## Câu 6 — LLM-as-judge qua LiteLLM: dùng 2 model khác nhau hay 1 model chạy 2 lần khác temperature? Bất đồng thì quyết ra sao?

---

**Interviewer đang test:** bạn có phân biệt **variance** với **bias** không.

### Đáp án

**Chọn 2 model khác family. Không phải 1 model 2 temperature.**

Lý do: cùng một model chạy hai lần chỉ đo được **variance của chính nó** — hai lần chạy có **correlated error**, chúng sai cùng một kiểu vì cùng prior, cùng training data. Đồng thuận giữa chúng không phải bằng chứng về tính đúng, nó chỉ là bằng chứng về tính nhất quán. Hai model khác family thì lỗi ít tương quan hơn, nên đồng thuận mới mang thông tin.

Và nguyên tắc bắt buộc: **judge phải cross-family với generator**. Nếu generator là Claude thì judge không được là Claude — nếu không sẽ dính self-preference bias. Ở LAAF tôi cố định `judge_model` cross-family và đưa nó vào fingerprint.

**Temperature 0 cho judge, luôn.** Randomness không phải là diversity. Judge cần **reproducible** — chấm lại cùng một câu ngày mai phải ra cùng kết quả, nếu không thì không so sánh được giữa các lần chạy eval.

### Cơ chế giải bất đồng — theo thứ tự:

1. **Deterministic tie-break trước hết.** Nếu ca đó có scorer deterministic thì **nó quyết, judge không tham gia**. Bất biến: LLM không lật deterministic.
2. **Escalate, không vote.** Hai judge bất đồng = tín hiệu ca đó khó hoặc **rubric mơ hồ**. Nó vào hàng đợi human review. Majority vote với 2 judge là vô nghĩa (không có đa số), và thêm judge thứ 3 để bẻ hoà chỉ là **nguy trang cho một rubric kém** — bạn đang mua thêm token để che một vấn đề định nghĩa.
3. **Fail-closed nếu buộc phải tự động.** Khi đây là ship gate và không thể chờ người: **bất đồng = FAIL**. Vì cost của việc ship sai lớn hơn cost của việc chặn nhầm.
4. **Ghi disagreement rate như metric của chính rubric.** Disagreement rate tăng theo thời gian = rubric đang drift hoặc distribution đầu vào đã đổi → **sửa rubric, không sửa judge**. Đây là chỗ hầu hết hệ thống bỏ sót: họ đo agent, không ai đo cái thước.

**Đo chất lượng judge đúng cách:** bằng **Cohen's kappa** với nhãn người trên một mẫu — không phải bằng "hai judge có giống nhau không". Hai judge cùng sai vẫn giống nhau.

### Follow-up về LiteLLM

LiteLLM cho phép đổi provider chỉ bằng config — tiện, nhưng có một cái bẫy: **provider âm thầm cập nhật model sau một alias**. Nếu `gpt-4o` hôm nay khác `gpt-4o` tháng trước thì mọi so sánh lịch sử của bạn hỏng mà không có tín hiệu nào. Nên tôi **pin model version cụ thể** và đưa `judge_model` + `rubric_version` + hash prompt judge vào **fingerprint của experiment**. Không có fingerprint thì không gọi là eval, chỉ là chấm điểm một lần.

---

## Câu 7 — Claude, GPT, Llama, DeepSeek: model nào cho phần nào trong Fabrion?

⚠️ **Đối chiếu lại với repo trước khi phỏng vấn lần sau.** Chỉ nói model bạn *thực sự* đã chạy. Bịa tên model là rủi ro lớn nhất — interviewer sẽ hỏi "chạy trên hạ tầng nào, tokens/s bao nhiêu, cost ra sao" và câu bịa sẽ vỡ ngay ở follow-up thứ nhất.

**Interviewer đang test:** bạn chọn model có lý do hay chọn theo hype.

### Khung trả lời — nói theo vai trò trong harness, không theo tên model

"Tôi không chọn model theo bảng xếp hạng, tôi chọn theo **vai trò trong harness**, và mỗi vai trò có một ràng buộc khác nhau."

| Vai trò            | Ràng buộc                                                                                        | Model đã dùng                                                                                                                                                              |
|--------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Agent-under-test   | Phải <b>đa dạng</b> — đó chính là mục đích của adapter seam N×M. Đổi vendor = đổi 1 file adapter | <code>gpt-oss-120b</code> trên NVIDIA/pi.dev cho task <code>build_dbt</code> / <code>build_cube</code> ; adapter chạy được cả pi.dev và OpenAI Codex CLI trên cùng bộ task |
| Judge              | <b>Cross-family với generator</b> , cố định, temp 0, nằm trong fingerprint                       | Cross-family DeepSeek-R1 / Claude                                                                                                                                          |
| Runtime chat agent | Cân bằng cost/latency, ground trên Cube semantic layer                                           | ( <i>điền model thật</i> )                                                                                                                                                 |
| Dev loop / tooling | Rẻ, nhanh, không vào kết quả eval                                                                | ( <i>điền model thật</i> )                                                                                                                                                 |

**Câu chốt:** "Bất biến của tôi là judge không bao giờ cùng family với generator. Và model nào cũng phải nằm trong fingerprint của experiment — vì đổi model mà không ghi lại thì mọi so sánh trước đó thành vô nghĩa."

**Nếu bị hỏi "sao không dùng model mạnh nhất cho mọi thứ?"** → Vì mục tiêu của harness không phải là đạt điểm cao nhất, mà là **phân biệt được các model với nhau**. Nếu tôi chạy model mạnh nhất ở mọi vị trí, tôi mất khả năng đo. Và với judge, mạnh hơn không đồng nghĩa đúng hơn — cùng family với generator thì càng mạnh càng thiên vị.

## Câu 8 — NL-to-SQL agent và LAAF: xây bằng LangChain hay tự viết hoàn toàn?

---

**Interviewer đang test:** bạn có biết *khi* nào nên dùng framework không. Cả hai câu trả lời cực đoan ("dùng hết framework" / "tự viết hết") đều là câu trả lời yếu.

### Đáp án

Tôi chia theo ranh giới rõ: **framework ở tầng runtime, tự viết ở tầng eval.**

**Runtime agent — dùng framework (Flowise/LangChain).** Ở đây framework tiết kiệm thời gian thật: tool binding, message formatting, streaming, retry. Không có lý do gì tự viết lại.

**Eval harness (LAAF) — tự viết hoàn toàn, và đây là quyết định có chủ đích:**

1. **Reproducibility là yêu cầu số một.** Harness phải fingerprint được: mã scorer, phiên bản prompt, model, seed, phiên bản bộ đề. Framework tự động thay đổi hành vi giữa các minor version — điều đó **âm thầm làm hỏng khả năng so sánh giữa các lần chạy**, mà không có tín hiệu nào. Với một hệ thống mà toàn bộ giá trị nằm ở việc so sánh, đó là rủi ro không chấp nhận được.
2. **Grader phải là code thuần.** `grade_marts.py`, `grade_cube.py`, `check_columns.py` là Python thuần, không LLM, không framework. Đó là điều kiện để nó được gọi là **source of truth**. Một grader phụ thuộc framework là một grader có thể đổi ý.
3. **Adapter seam.** Tôi cần đánh giá cả agent của người khác — pi.dev, Codex CLI — những thứ không nằm trong LangChain. Ma trận  $N \text{ model} \times M \text{ task}$  chỉ hoạt động khi ranh giới là **một file adapter**, không phải một abstraction của framework.
4. **Workspace ephemeral.** Mỗi trial copy workspace sạch. Framework mang state ẩn — cache, memory, callback — và state ẩn là kẻ thù của eval.

**Câu chốt:** "Tôi dùng framework ở chỗ nó tiết kiệm thời gian, và tự viết ở chỗ tôi cần kiểm soát tuyệt đối. Ranh giới là: thứ gì tham gia vào quyết định PASS/FAIL thì tôi viết."

---

## Câu 9 — Đã dùng LangChain chưa? LangChain vốn không mạnh về state machine phức tạp so với LangGraph — bạn có gặp giới hạn đó chưa?

**Interviewer đang test:** câu này có mồi. Họ đang mời bạn nói "có, nên tôi chuyển sang LangGraph". Trả lời hay hơn là chỉ ra bạn **gặp đúng vấn đề đó và giải theo cách khác, có lý do**.

### Đáp án

**Có, và giới hạn rất cụ thể.** LangChain kiểu chain/AgentExecutor cho bạn control flow tuyến tính với một vòng lặp ẩn bên trong. Với quy trình nhiều bước, tôi vấp đúng bốn chỗ:

| Giới hạn                                  | Hệ quả thực tế                                                                  |
|-------------------------------------------|---------------------------------------------------------------------------------|
| State ngầm định, không phải first-class   | Không biểu diễn được state machine có nhánh; logic nhánh rơi vào prompt         |
| Không checkpoint / resume                 | Fail ở bước 7 phải chạy lại từ bước 1 — tốn tiền, và mất tính tái lập           |
| Chèn human-in-the-loop giữa chừng rất khó | Phải cắt chain làm đôi, tự quản state ở giữa                                    |
| Retry ở mức chain, không mức node         | Không retry riêng một bước; observability cũng ở mức chain, khó quy lỗi về node |

LangGraph giải đúng bốn chỗ đó: state là first-class, conditional edge, checkpointer để resume, interrupt cho HITL, retry per-node.

**Nhưng đây là chỗ tôi muốn nói rõ quan điểm:**

"Recovery loop của tôi ở LAAF **vốn đã là một state machine viết tay** — grader reason → taxonomy tag ( `WRONG_ROWCOUNT` → *Grain Mismatch*, `EXECUTION_ERROR` → *Contract Violation*, `WRONG_VALUES` → *Logic Error*) → dựng prompt Round-2, tối đa 2 vòng, **và không bao giờ inject gold answer**. Tôi cố tình **không** đưa nó vào framework: Recovery Controller và Safety Gate là **code, không có model**, vì tôi cần chúng deterministic và audit được từng nhánh. Một eval controller chạy trên framework là một eval controller có thể đổi hành vi khi framework lên version."

Nên kết luận của tôi có hai vế:

- **Runtime agent nhiều nhánh, có HITL, cần resume** → LangGraph là lựa chọn đúng, tôi sẽ dùng.
- **Eval controller** → tự viết, vì reproducibility và audit quan trọng hơn tiện lợi.

### Nếu chưa dùng LangGraph trong production — nói thẳng

"Tôi chưa chạy LangGraph trong production. Tôi gặp đúng lớp vấn đề mà nó giải, và tôi giải bằng một controller tự viết vì ràng buộc reproducibility của harness. Nếu team đang xây runtime agent nhiều nhánh thì tôi thấy LangGraph là lựa chọn hợp lý và tôi lên tay nhanh — mô hình state/edge/checkpoint không xa lạ với thứ tôi đã tự viết."

Trung thực ở đây **mạnh hơn** giả vờ đã dùng. Interviewer hỏi câu này thường là người đã dùng thật, và họ sẽ hỏi tiếp về checkpointer backend hoặc cách xử lý interrupt.

## Câu 10 — Làm rõ pass rate 0.14 → 0.93. Bày deterministic scorer là gì?

**Interviewer đang test:** con số này có thật không, và bạn có hiểu nó không. Đây là con số chủ lực của bạn — phải kể được như một quá trình, không phải như một thành tích.

### Đáp án — kể theo phương pháp

**Bối cảnh:** NL-to-SQL agent doanh nghiệp, benchmark **BEAVER 209 câu**, versioned và **không lộ cho agent**. Composite pass rate qua **33 vòng** diagnose → fix → re-test: **0.14 → 0.93**.

**Giải thích ngay vì sao 0.14 không có nghĩa là "agent hầu như không chạy":** đây là **composite** — hợp của nhiều scorer, và một scorer fail là kéo cả composite xuống. Agent lúc đó trả lời được nhiều câu, nhưng trượt ở cột sai tên, sai grain, hoặc query không tái lập được. Composite khắt khe là **cố ý**: tôi muốn một con số mà không thể đẹp lên bằng cách trả lời chung chung.

### Mức tăng đến từ đâu — đây là phần quan trọng nhất:

Phần lớn **không** đến từ đổi model hay "prompt tốt hơn". Nó đến từ ba thứ, xếp theo mức đóng góp:

- Ground lên Cube semantic layer thay vì để agent tự viết SQL.** Đây là thay đổi lớn nhất — nó **loại bỏ nguyên một lớp lỗi**: join sai, grain sai, định nghĩa metric không nhất quán. Metric đã được định nghĩa một lần ở semantic layer thì agent không còn cơ hội định nghĩa lại nó sai.
- Sửa tool description và tool schema.** Tool description chính là prompt. Một phần đáng kể lỗi ban đầu là chọn sai tool / sai tham số, không phải reasoning kém.
- Contract validation trước khi execute** (dry-run) — biến lỗi im lặng thành lỗi có cấu trúc, và lỗi có cấu trúc thì recovery loop xử lý được.

**Phương pháp mỗi vòng:** không sửa theo cảm nhận. Mỗi vòng là: chạy toàn bộ 209 câu → grader trả về **reason code** → gom thành **failure-mode distribution** → sửa nhóm lớn nhất trước → chạy lại toàn bộ → so với baseline. Nguyên tắc: **thiết lập phân bố lỗi TRƯỚC khi can thiệp**. Sửa trước khi đo là cách nhanh nhất để tốn 33 vòng mà không tiến.

**Tính trung thực — nói ra chủ động:** trong quá trình này tôi đã **rút 3 kết quả** vì xác định được chúng là artefact hạ tầng/randomness chứ không phải cải thiện thật. Và mọi con số đều có **null control + cheater control** — agent trả lời rỗng và agent gian lận đều phải được 0 điểm. Nếu không có hai control đó thì 0.93 không kiểm chứng được.

### Bày scorer

**⚠** Đối chiếu tên hàm với repo trước khi phỏng vấn. Bốn cái đầu là những cái bạn đã nêu trong tài liệu cũ; ba cái sau từ kiến trúc LAAF.

| # | Scorer                                                                                    | Đo gì                                                                                        | Kiểu             |
|---|-------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|------------------|
| 1 | SQL execution equivalence<br>( <code>grade_marts.py</code> / <code>grade_cube.py</code> ) | Chạy <b>cả gold và candidate</b> , so kết quả thực thi                                       | Liên tục (ratio) |
| 2 | Rowset semantics                                                                          | So tập hàng <b>không phụ thuộc thứ tự hàng/cột</b> , có tolerance cho float, xử lý NULL đúng | Liên tục         |
| 3 | Column contract ( <code>check_columns.py</code> )                                         | Tên cột, kiểu dữ liệu, số cột đúng hợp đồng                                                  | Nhị phân         |
| 4 | Tool-call correctness                                                                     | Gọi đúng tool, đúng tham số, đúng thứ tự                                                     | Nhị phân         |

| # | Scorer                      | Đo gì                                                    | Kiểu                                                    |
|---|-----------------------------|----------------------------------------------------------|---------------------------------------------------------|
| 5 | Answer grounding            | Mọi con số trong câu trả lời truy được về rowset thực tế | Nhị phân                                                |
| 6 | Safety Gate $G \in \{0,1\}$ | Vi phạm an toàn/ràng buộc cứng                           | <b>Nhân với tổng</b> — vi phạm là 0 bất kể phần còn lại |
| 7 | Non-regression              | So với baseline đã pass trước đó, không được tụt         | Nhị phân, chạy trong CI                                 |

Công thức tổng:  $\text{Total} = G \times (W1 \cdot \text{EPQ} + W2 \cdot \text{EQ})$ .  $G$  nhân chứ không cộng — đó là cách diễn đạt "an toàn không đánh đổi được bằng độ chính xác".  $\text{EQ}$  (execution quality) dùng **cả hai dạng**: ngưỡng nhị phân  $\text{ratio} \geq 0.85$  để quyết ship/no-ship, và ratio liên tục cho leaderboard.

**Follow-up gần như chắc chắn**

"**Sao không so SQL string cho nhanh?**" → Vì hai câu SQL trông hoàn toàn khác nhau vẫn có thể đúng như nhau, và hai câu gần giống nhau có thể cho kết quả khác nhau. **So text là một scorer sai** — nó phạt agent vì viết khác gold, chứ không thưởng agent vì trả lời đúng. Exec-equivalence đắt hơn (phải chạy thật, phải có warehouse tái lập được) nhưng nó đo đúng thứ mình quan tâm.

"**Làm sao biết grader của bạn đúng?**" → Ba cách: gold-as-candidate phải ra 1.0; null và cheater phải ra 0; và **mutant test** — cố tình làm hỏng implementation thì grader phải fail. Grader không được kiểm bằng control thì cũng chỉ là một ý kiến.

---

Câu 11 — Case study: RAG/Multi-agent cho 100 khách hàng, mỗi khách có bộ data riêng. Thiết kế isolation ở tầng dữ liệu, vector database, agent context?

Interviewer đang test: system design + bạn có nghĩ tới cross-tenant leak không.

Mở đầu — hỏi lại trước khi vẽ (chữ ký của bạn)

"Trước khi vẽ, tôi cần 3 thông tin vì chúng đổi hẵn kiến trúc: (1) yêu cầu compliance là **logical isolation** hay **bắt buộc physical/data residency**? (2) phân bố dữ liệu giữa 100 tenant lệch bao nhiêu — có tenant nào chiếm 80% không? (3) tenant có được **custom schema** hay dùng chung một mô hình dữ liệu?"

Và câu tôi luôn nói trước: **định nghĩa metric trước kiến trúc**. Với bài này metric quan trọng nhất không phải latency mà là **cross-tenant leak rate** — và nó phải bằng 0, đo được trong CI.

Ba mô hình, và tôi chọn gì

| Mô hình                                                 | Isolation | Cost/vận hành                              | Phù hợp                                      |
|---------------------------------------------------------|-----------|--------------------------------------------|----------------------------------------------|
| Silo — DB riêng mỗi tenant                              | Mạnh nhất | Cao: vận hành 100 stack, migration 100 lần | Tenant enterprise có yêu cầu compliance cứng |
| Pool — chung hết, phân biệt bằng <code>tenant_id</code> | Yếu nhất  | Rẻ nhất                                    | Long tail nhỏ, nhưng rủi ro leak cao         |
| Bridge — cluster chung, <b>schema/namespace riêng</b>   | Tốt       | Vừa                                        | ✅ Mặc định của tôi cho 100 tenant            |

Tôi chọn **bridge làm mặc định, silo cho tenant có yêu cầu compliance cứng** — tức là kiến trúc phải hỗ trợ cả hai từ đầu, chọn per-tenant bằng config.

Isolation ở 5 tầng — và cả 5 phải đúng

1. Data layer

ClickHouse cluster chung, **database riêng mỗi tenant**. Role warehouse riêng mỗi tenant, read-only. Row-level security làm lớp phòng thủ thứ hai.

Nguyên tắc quan trọng nhất: **không bao giờ dựa vào `WHERE tenant_id = ...` do LLM sinh ra**. Bộ lọc tenant phải được **inject ở tầng dưới LLM** — trong query builder hoặc semantic layer — nơi prompt injection không với tới. Nếu isolation của bạn nằm trong một câu SQL mà model viết, thì bạn không có isolation.

2. Vector database

**Collection/namespace riêng mỗi tenant** (Qdrant collection, Pinecone namespace, pgvector schema + RLS). Không dùng một index chung lọc bằng metadata.

Lý do dứt khoát: metadata filter là **runtime** — một bug ở filter là leak toàn bộ, im lặng, và bạn chỉ biết khi khách hàng phát hiện. Namespace là **structural** — muốn leak phải gọi sai tên collection, và cái đó bắt được bằng test.

Thêm hai chi tiết vận hành: **whale tenant tách collection riêng** để volume của họ không kéo tụt recall/latency của người khác; và **pin embedding model version per-tenant** để có thể re-index cuốn chiếu chứ không phải đồng loạt 100 tenant một lúc.

3. Agent context

- `tenant_id` **resolve từ auth token, không bao giờ từ nội dung câu hỏi**. Người dùng gõ "cho tôi xem số của công ty B" không được đổi được tenant.

- `tenant_id` đi vào **request context** và **bind** vào tool qua **dependency injection/closure** — **không bao giờ là một argument mà LLM điền**. Model phải **không có khả năng vật lý** để đặt `tenant_id`. Đây là điểm thiết kế quan trọng nhất của cả câu trả lời.
- System prompt, semantic layer, glossary, few-shot examples: per-tenant, versioned.

#### 4. Cache và memory — chỗ leak kinh điển bị bỏ sót

Mọi cache key **phải chứa `tenant_id`**: semantic cache, embedding cache, query result cache, prompt cache prefix. Conversation memory store riêng. Một semantic cache không phân tenant sẽ trả câu trả lời của tenant A cho tenant B, và nó **trông hoàn toàn bình thường trong log**.

#### 5. Eval và observability

Đây là chỗ tôi mang chữ ký riêng vào:

- Golden set nhỏ **per-tenant** (10–20 câu) để bắt regression riêng của từng tenant.
- **Cross-tenant leak test chạy trong CI** — hỏi agent của tenant A một câu mà **chỉ dữ liệu tenant B trả lời được**. Agent phải abstain. Bất kỳ câu trả lời có nội dung nào = **fail build**. Đây chính là control kiểu null/cheater áp cho multi-tenancy.
- Trace gắn `tenant_id`; **cost per tenant là một SLI**, không phải con số cuối tháng.

#### Câu chốt

"Isolation không phải một lớp — nó là một bất biến phải đúng ở **5 chỗ**: warehouse, vector store, tool argument, cache, và trace. **Chỗ yếu nhất quyết định mức isolation thật của hệ thống**. Và cách duy nhất tôi tin nó đúng là có một leak test chạy mỗi lần CI, chứ không phải đọc lại code."

## Câu 12 — Có 10 tool, LLM chọn sai tool. Xử lý thế nào để cải thiện chất lượng?

Interviewer đang test: bạn nhảy vào prompt engineering hay bạn đo trước.

### Đáp án

**Bước 0 — đo trước khi sửa.** Tôi dựng một **confusion matrix của tool**: tool đúng × tool được chọn, trên bộ đề có nhãn. Không có ma trận này thì đang sửa mù — và thường sẽ phát hiện ra 80% lỗi tập trung ở **1–2 cặp tool bị nhầm lẫn với nhau**, chứ không rải đều 10 tool. Sửa đúng 2 cặp đó rẻ hơn viết lại toàn bộ prompt rất nhiều.

**Bước 1 — phân loại nguyên nhân**, vì mỗi loại có cách sửa khác nhau:

| Nguyên nhân                         | Dấu hiệu trong confusion matrix           | Cách sửa                                                                                  |
|-------------------------------------|-------------------------------------------|-------------------------------------------------------------------------------------------|
| Hai tool <b>chồng lấn chức năng</b> | Nhầm đối xứng: A→B và B→A đều nhiều       | <b>Gộp lại</b> , hoặc phân định ranh giới rõ trong description                            |
| Description mơ hồ                   | Một tool bị chọn cho mọi thứ (tool "rác") | Viết lại: "dùng khi... / KHÔNG dùng khi..."                                               |
| Thiếu tool phù hợp                  | Rải đều, không có pattern                 | Model chọn cái gần nhất vì không có cái đúng → thêm tool, hoặc thêm đường abstain         |
| Quá nhiều tool                      | Độ chính xác tụt khi tăng số tool         | Tool search / defer loading, hoặc <b>router 2 tầng</b> : chọn nhóm → chọn tool trong nhóm |
| Sai tham số, không sai tool         | Tool đúng nhưng kết quả sai               | Strict schema + <code>enum</code> , không phải vấn đề chọn tool                           |

**Bước 2 — sửa theo thứ tự ROI:**

- Viết lại tool description** cho các cặp hay nhầm, kèm "dùng khi / không dùng khi" và 1–2 ví dụ. **Rẻ nhất, hiệu quả nhất.** Tool description chính là prompt — sai tool thường là lỗi mô tả tool, không phải lỗi model.
- Strict JSON schema:** `enum` thay vì free string, `additionalProperties: false`, validate trước khi thực thi, và **trả lỗi có cấu trúc** để model tự sửa ở turn sau thay vì thất bại im lặng.
- Gộp tool trùng chức năng** — 10 tool thường thực ra chỉ nên là 6. Giảm số lựa chọn là cách cải thiện độ chính xác chọn tool đáng tin cậy nhất.
- Few-shot ví dụ tool-call** cho đúng các cặp hay nhầm (không cần cho cả 10).
- Hierarchical routing / tool search** khi số tool tăng lên hàng chục.
- Chỉ khi vẫn sai và đã có đủ dữ liệu có nhãn: **router model nhỏ chuyên chọn tool**, hoặc fine-tune. Đây là bước cuối, không phải bước đầu.

**Bước 3 — guardrail: chọn sai tool không được biến thành hành động sai.**

Read-only mặc định, dry-run cho write, tool sai trả lỗi có cấu trúc thay vì im lặng. Một agent chọn sai tool nhưng bị chặn ở lớp thực thi thì đó là một lần retry; chọn sai tool mà thực thi được thì đó là một sự cố.

**Câu chốt:**

"Tool-call correctness là một trong 7 scorer deterministic của tôi. Nên tôi coi đây là bài toán đo lường, không phải bài toán prompt engineering: dựng confusion matrix → khoanh vùng cặp nhầm → sửa description hoặc gộp tool → chạy lại toàn bộ bộ đề → và khoá lại bằng non-regression gate để nó không quay lại."

## Rút kinh nghiệm sau buổi CDC Global

---

Đọc kèm phần **Bối cảnh — 12 bộ chuẩn dữ liệu cốt lõi** ở đầu tài liệu: bốn chủ đề dưới đây map gần như một-một vào các bộ chuẩn số 2, 6, 8, 10 và 12.

**Bốn chủ đề họ đào sâu** — đây là chân dung thứ họ cần:

1. **Chẩn đoán RAG có phương pháp** (câu 1, 2, 3) — họ đang có hoặc sắp có RAG production và đã bị đau.
2. **Guardrail cho agent có quyền hành động** (câu 4, 12) — họ định cho agent gọi API thật.
3. **Eval đáng tin** (câu 5, 6) — hai câu liên tiếp về LLM-as-judge, khả năng cao họ đang có khoảng cách giữa số eval và cảm nhận người dùng. **Đây là sân nhà của bạn.**
4. **Multi-tenant** (câu 11) — sản phẩm B2B nhiều khách hàng.

**Ba điều cần chuẩn bị kỹ hơn cho lần sau:**

- **Bảng model x vai trò** (câu 7): viết ra giấy tên model thật cho từng vị trí trong Fabrian, kèm lý do chọn và cost. Đây là câu duy nhất bạn không thể ứng biến an toàn.
- **Bảng scorer**: học thuộc tên + đo gì + kiểu (nhị phân/liên tục), và câu trả lời cho "*sao không so SQL string*" và "*làm sao biết grader đúng*".
- **LangGraph**: nếu chưa dùng thật thì chuẩn bị câu trung thực ở Câu 9 — nói được checkpoint và interrupt giải vấn đề gì, và vì sao Recovery Controller của bạn cố ý không dùng nó.

**Câu hỏi ngược nên hỏi họ** (rất khớp với những gì họ đã hỏi):

1. "Hiện tại pass rate mà team báo cáo đến từ deterministic scorer hay từ LLM judge?"
2. "Có golden set và regression gate trước khi ship không, hay eval chạy sau khi ship?"
3. "Agent đã được cấp quyền write ở đâu chưa, và ai duyệt?"
4. "Với multi-tenant, có leak test tự động trong CI không?"

Nếu họ không trả lời được câu 1 và 2 — **đó chính là công việc của bạn ở đó, và nên nói thẳng.**